FLET FRAMEWORK COMPLETE REFERENCE GUIDE

Table of Contents

1. [Introduction to Flet](#1-introduction-to-flet)
2. [Adaptive Apps](#2-adaptive-apps)
3. [Keyboard Shortcuts & Events](#3-keyboard-shortcuts--events)
4. [Drag and Drop](#4-drag-and-drop)
5. [Large Lists & Performance](#5-large-lists--performance)
6. [Accessibility](#6-accessibility)
7. [Navigation and Routing](#7-navigation-and-routing)
8. [Router System](#8-router-system)
9. [Control Refs](#9-control-refs)
10. [Custom Controls](#10-custom-controls)
11. [Declarative vs Imperative](#11-declarative-vs-imperative)
12. [Async Apps](#12-async-apps)
13. [File Operations](#13-file-operations)
14. [Client Storage](#14-client-storage)
15. [Session Storage](#15-session-storage)
16. [PubSub System](#16-pubsub-system)
17. [Subprocess Operations](#17-subprocess-operations)
18. [Multiprocessing](#18-multiprocessing)
19. [Authentication](#19-authentication)
20. [Data Encryption](#20-data-encryption)
21. [Declarative Dialogs](#21-declarative-dialogs)
22. [Flet MCP Server](#22-flet-mcp-server)
23. [Android Packaging](#23-android-packaging)
24. [Publishing Web Apps](#24-publishing-web-apps)
25. [Auto-Update & Logging](#25-auto-update--logging)

---

1. Introduction to Flet

Flet is a framework that allows you to build interactive web, desktop, and mobile applications in Python. It's based on Flutter and provides over 140+ built-in controls.

Core Concepts

- **Page**: The main container for your app's UI
- **Controls**: UI elements like buttons, text fields, etc.
- **Event Handlers**: Functions that respond to user interactions
- **State Management**: Managing data that affects your UI

Basic App Structure

```python
import flet as ft
```

```
def main(page: ft.Page):
    # Create and configure controls
    text = ft.Text("Hello, World!")
    page.add(text)

ft.run(main)
```

Key Features

- **Single Codebase**: Write once, deploy to web, desktop, and mobile
- **Adaptive UI**: Automatically adapts to platform (iOS/Android)
- **Rich Controls**: 140+ built-in controls
- **Async Support**: Build responsive apps with async/await
- **Hot Reload**: See changes instantly during development
- **State Management**: Supports both imperative and declarative styles

---

2. Adaptive Apps

Flet allows you to create apps that automatically adapt their look and feel based on the platform (iOS, Android, web, desktop).

Enabling Adaptive Mode

```python
import flet as ft

def main(page: ft.Page):
    page.adaptive = True  # Enable adaptive mode

    page.appbar = ft.AppBar(
        leading=ft.TextButton("New", style=ft.ButtonStyle(padding=0)),
        title=ft.Text("Adaptive AppBar"),
        actions=[
            ft.IconButton(ft.CupertinoIcons.ADD, style=ft.ButtonStyle(padding=0))
        ],
        bgcolor=ft.Colors.with_opacity(0.04, ft.CupertinoColors.SYSTEM_BACKGROUND),
    )

    page.navigation_bar = ft.NavigationBar(
        destinations=[
            ft.NavigationBarDestination(icon=ft.Icons.EXPLORE, label="Explore"),
            ft.NavigationBarDestination(icon=ft.Icons.COMMUTE, label="Commute"),
            ft.NavigationBarDestination(
                icon=ft.Icons.BOOKMARK_BORDER,
                selected_icon=ft.Icons.BOOKMARK,
                label="Bookmark",
```

```
            ),
        ],
        border=ft.Border(
            top=ft.BorderSide(color=ft.CupertinoColors.SYSTEM_GREY2, width=0)
        ),
    )

    page.add(
        ft.SafeArea(
            ft.Column([
                ft.Checkbox(value=False, label="Dark Mode"),
                ft.Text("First field:"),
                ft.TextField(keyboard_type=ft.KeyboardType.TEXT),
                ft.Text("Second field:"),
                ft.TextField(keyboard_type=ft.KeyboardType.TEXT),
                ft.Switch(label="A switch"),
                ft.FilledButton(content=ft.Text("Adaptive button")),
                ft.Text("Text line 1"),
                ft.Text("Text line 2"),
                ft.Text("Text line 3"),
            ])
        )
    )

ft.run(main)
```

Material vs Cupertino Controls

- **Material Controls**: Google's design system (default for Android)
- **Cupertino Controls**: Apple's design system (default for iOS)

Adaptive Property

Controls with `adaptive=True` automatically switch between Material and Cupertino versions:

```python
# Automatically shows CupertinoCheckbox on iOS, Material Checkbox on Android
ft.Checkbox(adaptive=True, value=True, label="Adaptive Checkbox")
```

Platform Detection

```python
from flet import PagePlatform

if page.platform == PagePlatform.IOS or page.platform == PagePlatform.MACOS:
    # iOS/macOS specific code
else:
```

```
    # Android/other platform code
```

Custom Adaptive Controls

Create your own adaptive controls by checking platform in the `build()` method:

```python
import flet as ft

class AdaptiveNavigationBarDestination(ft.NavigationBarDestination):
    def __init__(self, ios_icon, android_icon, label):
        super().__init__()
        self._ios_icon = ios_icon
        self._android_icon = android_icon
        self.label = label

    def build(self):
        self.icon = (
            self._ios_icon
            if self.page.platform == ft.PagePlatform.IOS
            or self.page.platform == ft.PagePlatform.MACOS
            else self._android_icon
        )

def main(page: ft.Page):
    page.adaptive = True

    page.navigation_bar = ft.NavigationBar(
        selected_index=2,
        destinations=[
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.PERSON_3_FILL,
                android_icon=ft.Icons.PERSON,
                label="Contacts",
            ),
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.CHAT_BUBBLE_2,
                android_icon=ft.Icons.CHAT,
                label="Chats",
            ),
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.SETTINGS,
                android_icon=ft.Icons.SETTINGS,
                label="Settings",
            ),
        ],
    )
    page.update()
```

```
ft.run(main)
```

---

3. Keyboard Shortcuts & Events

Flet provides comprehensive keyboard support for web and desktop apps.

Global Keyboard Events

```python
import flet as ft

def main(page: ft.Page):
    def on_keyboard(e: ft.KeyboardEvent):
        page.add(
            ft.Text(
                f"Key: {e.key}, Shift: {e.shift}, Control: {e.ctrl}, "
                f"Alt: {e.alt}, Meta: {e.meta}"
            )
        )

    page.on_keyboard_event = on_keyboard
    page.add(ft.Text("Press any key with combinations..."))

ft.run(main)
```

KeyboardEvent Properties

Property	Type	Description
`key`	str	Textual representation (e.g., "A", "Enter", "F5")
`shift`	bool	Shift key pressed
`ctrl`	bool	Control key pressed
`alt`	bool	Alt/Option key pressed
`meta`	bool	Command key pressed (macOS)

Advanced Keyboard Example

```python
import flet as ft

class ButtonControl(ft.Container):
    def __init__(self, text):
        super().__init__()
        self.content = ft.Text(text)
```

```
        self.border = ft.Border.all(1, ft.Colors.BLACK_54)
        self.border_radius = 3
        self.bgcolor = "0x09000000"
        self.padding = 10
        self.visible = False

def main(page: ft.Page):
    page.spacing = 50
    page.vertical_alignment = ft.MainAxisAlignment.CENTER
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    def on_keyboard(e: ft.KeyboardEvent):
        key.content.value = e.key
        key.visible = True
        shift.visible = e.shift
        ctrl.visible = e.ctrl
        alt.visible = e.alt
        meta.visible = e.meta
        page.update()

    page.on_keyboard_event = on_keyboard

    page.add(
        ft.SafeArea(
            content=ft.Column([
                ft.Text(
                    "Press any key with a combination of CTRL, ALT, SHIFT "
                    "and META keys..."
                ),
                ft.Row([
                    key := ButtonControl(""),
                    shift := ButtonControl("Shift"),
                    ctrl := ButtonControl("Control"),
                    alt := ButtonControl("Alt"),
                    meta := ButtonControl("Meta"),
                ], alignment=ft.MainAxisAlignment.CENTER),
            ])
        )
    )

ft.run(main)
```

Focus Management

```python
# Auto-focus a control
text_field = ft.TextField(autofocus=True)
```

```
# Programmatically focus
await text_field.focus()
```

---

4. Drag and Drop

Flet's drag-and-drop system uses `Draggable` and `DragTarget` controls.

Basic Drag and Drop

```python
import flet as ft

def main(page: ft.Page):
    page.title = "Drag and Drop example"

    def drag_accept(e):
        # Update source control
        e.src.content.content.value = "0"
        # Update target control
        e.control.content.content.value = "1"
        page.update()

    page.add(
        ft.Row([
            ft.Draggable(
                group="number",
                content=ft.Container(
                    width=50,
                    height=50,
                    bgcolor=ft.Colors.CYAN_200,
                    border_radius=5,
                    content=ft.Text("1", size=20),
                    alignment=ft.Alignment.CENTER,
                ),
            ),
            ft.Container(width=100),
            ft.DragTarget(
                group="number",
                content=ft.Container(
                    width=50,
                    height=50,
                    bgcolor=ft.Colors.PINK_200,
                    border_radius=5,
                    content=ft.Text("0", size=20),
                    alignment=ft.Alignment.CENTER,
                ),
```

```
                on_accept=drag_accept,
            ),
        ])
    )

ft.run(main)
```

Draggable Properties

- `group`: Group identifier (drag targets only accept from same group)
- `content`: Control to display normally
- `content_when_dragging`: Control shown in original position while dragging
- `content_feedback`: Control shown under cursor while dragging

DragTarget Events

```python
def drag_will_accept(e):
    # e.data == "true" if target can accept the drag
    e.control.content.border = ft.Border.all(
        2, ft.Colors.BLACK_45 if e.data == "true" else ft.Colors.RED
    )
    e.control.update()

def drag_leave(e):
    e.control.content.border = None
    e.control.update()

def drag_accept(e):
    # Handle successful drop
    pass

drag_target = ft.DragTarget(
    group="number",
    content=...,
    on_accept=drag_accept,
    on_will_accept=drag_will_accept,
    on_leave=drag_leave,
)
```

Complete Example with Visual Feedback

```python
import flet as ft

def main(page: ft.Page):
    page.title = "Drag and Drop example 2"
```

```
def drag_accept(e):
    e.src.content.content.value = "0"
    e.src.group = ""  # Disable after use
    e.control.content.content.value = "1"
    e.control.content.border = None
    page.update()

def drag_will_accept(e):
    e.control.content.border = ft.Border.all(
        2, ft.Colors.BLACK_45 if e.data == "true" else ft.Colors.RED
    )
    e.control.update()

def drag_leave(e):
    e.control.content.border = None
    e.control.update()

page.add(
    ft.Row([
        ft.Draggable(
            group="number",
            content=ft.Container(
                width=50,
                height=50,
                bgcolor=ft.Colors.CYAN_200,
                border_radius=5,
                content=ft.Text("1", size=20),
                alignment=ft.Alignment.CENTER,
            ),
            content_when_dragging=ft.Container(
                width=50,
                height=50,
                bgcolor=ft.Colors.BLUE_GREY_200,
                border_radius=5,
            ),
            content_feedback=ft.Text("1"),
        ),
        ft.Container(width=100),
        ft.DragTarget(
            group="number",
            content=ft.Container(
                width=50,
                height=50,
                bgcolor=ft.Colors.PINK_200,
                border_radius=5,
                content=ft.Text("0", size=20),
                alignment=ft.Alignment.CENTER,
            ),
```

```
                on_accept=drag_accept,
                on_will_accept=drag_will_accept,
                on_leave=drag_leave,
            ),
        ])
    )

ft.run(main)
```

---

5. Large Lists & Performance

For large lists, use `ListView` and `GridView` instead of `Column` or `Row`.

Problem with Column/Row for Large Lists

```python
import flet as ft

def main(page: ft.Page):
    # This will be slow with 5000 items
    for i in range(5000):
        page.controls.append(ft.Text(f"Line {i}"))
    page.scroll = "always"
    page.update()

ft.run(main, view=ft.AppView.WEB_BROWSER)
```

ListView (Virtual Scrolling)

```python
import flet as ft

def main(page: ft.Page):
    lv = ft.ListView(expand=True, spacing=10)
    for i in range(5000):
        lv.controls.append(ft.Text(f"Line {i}"))
    page.add(lv)

ft.run(main, view=ft.AppView.WEB_BROWSER)
```

ListView Performance Tips

```python
# Fixed item height for better performance
```

```
lv = ft.ListView(expand=True, spacing=10, item_extent=50)

# Or use first item as prototype
lv = ft.ListView(expand=True, spacing=10, first_item_prototype=True)
```

GridView

```python
import os
import flet as ft

os.environ["FLET_WS_MAX_MESSAGE_SIZE"] = "8000000"

def main(page: ft.Page):
    gv = ft.GridView(expand=True, max_extent=150, child_aspect_ratio=1)
    page.add(gv)

    for i in range(5000):
        gv.controls.append(
            ft.Container(
                ft.Text(f"Item {i}"),
                alignment=ft.Alignment.CENTER,
                bgcolor=ft.Colors.AMBER_100,
                border=ft.Border.all(1, ft.Colors.AMBER_400),
                border_radius=ft.BorderRadius.all(5),
            )
        )
    page.update()

ft.run(main, view=ft.AppView.WEB_BROWSER)
```

GridView Properties

- `max_extent`: Maximum size of each tile
- `child_aspect_ratio`: Width/height ratio (1 = square)
- `runs_count`: Fixed number of rows/columns

Batch Updates for Performance

```python
import flet as ft

def main(page: ft.Page):
    lv = ft.ListView(expand=1, spacing=10, item_extent=50)
    page.add(lv)

    for i in range(5100):
```

```
        lv.controls.append(ft.Text(f"Line {i}"))
        # Update every 500 items
        if i % 500 == 0:
            page.update()

    # Final update for remaining items
    page.update()

ft.run(main, view=ft.AppView.WEB_BROWSER)
```

---

6. Accessibility

Flet supports accessibility features including screen readers.

Screen Reader Support

Mobile:
- iOS: VoiceOver
- Android: TalkBack

Desktop Browsers:
- macOS: VoiceOver
- Windows: JAWS & NVDA

Accessibility Properties

```python
# Text with custom semantics label
ft.Text("Hello", semantics_label="Greeting message")

# Button with tooltip for screen readers
ft.IconButton(ft.Icons.ADD, tooltip="Add new item")

# TextField with label for semantics
ft.TextField(label="Enter your name")

# Dropdown with label
ft.Dropdown(label="Select country")
```

Semantics Control for Custom Accessibility

```python
ft.Semantics(
    content=ft.Text("Custom element"),
    label="This is a custom accessible element",
```

```
    hint="Double tap to activate",
    enabled=True,
    checked=True,
    value="Custom value",
)
```

Debugging Semantics

```python
# Enable semantics debugger overlay
page.show_semantics_debugger = True

# Toggle with keyboard shortcut
def on_keyboard(e: ft.KeyboardEvent):
    if e.shift and e.key == "S":
        page.show_semantics_debugger = not page.show_semantics_debugger
        page.update()

page.on_keyboard_event = on_keyboard
```

Complete Accessibility Example

```python
import flet as ft

def main(page: ft.Page):
    page.vertical_alignment = ft.MainAxisAlignment.CENTER
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    def on_keyboard(e: ft.KeyboardEvent):
        if e.shift and e.key == "S":
            page.show_semantics_debugger = not page.show_semantics_debugger
            page.update()

    page.on_keyboard_event = on_keyboard

    def button_click(e):
        counter.value = str(int(counter.value) + 1)
        page.update()

    page.add(
        ft.SafeArea(
            content=ft.Column([
                counter := ft.Text("0", size=40),
                ft.Text("Press Shift+S to toggle semantics debugger"),
                ft.Button(
                    content="Increment number",
```

```
                icon=ft.Icons.ADD,
                on_click=button_click,
            ),
        ])
    )
)

ft.run(main)
```

---

7. Navigation and Routing

Flet's routing system allows building multi-screen apps with URL/history support.

Basic Routing

```python
import flet as ft

def main(page: ft.Page):
    page.add(ft.SafeArea(content=ft.Text(f"Initial route: {page.route}")))

ft.run(main)
```

Handling Route Changes

```python
import flet as ft

def main(page: ft.Page):
    route_log = ft.Column([ft.Text(f"Initial route: {page.route}")])
    page.add(ft.SafeArea(content=route_log))

    def route_change(e):
        route_log.controls.append(ft.Text(f"New route: {e.route}"))
        page.update()

    page.on_route_change = route_change
    page.update()

ft.run(main)
```

Building Views from Route

```python
```

```
import flet as ft

def main(page: ft.Page):
    page.title = "Routes Example"

    async def open_mail_settings(e):
        await page.push_route("/settings/mail")

    async def open_settings(e):
        await page.push_route("/settings")

    def route_change():
        page.views.clear()
        page.views.append(
            ft.View(
                route="/",
                controls=[
                    ft.SafeArea(
                        content=ft.Column([
                            ft.AppBar(title=ft.Text("Flet app")),
                            ft.Button("Go to settings", on_click=open_settings),
                        ])
                    )
                ],
            )
        )

        if page.route == "/settings" or page.route == "/settings/mail":
            page.views.append(
                ft.View(
                    route="/settings",
                    controls=[
                        ft.SafeArea(
                            content=ft.Column([
                                ft.AppBar(
                                    title=ft.Text("Settings"),
                                    bgcolor=ft.Colors.SURFACE_CONTAINER_HIGHEST,
                                ),
                                ft.Text("Settings!", theme_style=ft.TextThemeStyle.BODY_MEDIUM),
                                ft.Button("Go to mail settings", on_click=open_mail_settings),
                            ])
                        )
                    ],
                )
            )

        if page.route == "/settings/mail":
            page.views.append(
                ft.View(
```

```
                route="/settings/mail",
                controls=[
                    ft.SafeArea(
                        content=ft.Column([
                            ft.AppBar(
                                title=ft.Text("Mail Settings"),
                                bgcolor=ft.Colors.SURFACE_CONTAINER_HIGHEST,
                            ),
                            ft.Text("Mail settings!"),
                        ])
                    )
                ],
            )
        )
        page.update()

    async def view_pop(e):
        if e.view is not None:
            page.views.remove(e.view)
            top_view = page.views[-1]
            await page.push_route(top_view.route)

    page.on_route_change = route_change
    page.on_view_pop = view_pop
    route_change()

ft.run(main)
```

Programmatic Navigation

```python
# Synchronous (on_click, etc.)
page.navigate("/search", q="flet", page=2)

# Asynchronous
await page.push_route("/search", q="flet", page=2)
```

Pop Confirmation

```python
import flet as ft

class PermissionView(ft.View):
    def __init__(self, path):
        super().__init__(
            route=path,
            controls=[
```

```
                ft.SafeArea(
                    content=ft.Column([
                        ft.AppBar(title=ft.Text(f"{path} View"))
                    ])
                )
            ],
            can_pop=False,
            on_confirm_pop=self.ask_pop_permission,
        )

    async def ask_pop_permission(self, e):
        async def on_dlg_yes(e):
            self.page.pop_dialog()
            await self.confirm_pop(True)

        async def on_dlg_no(e):
            self.page.pop_dialog()
            await self.confirm_pop(False)

        dlg_modal = ft.AlertDialog(
            title=ft.Text("Please confirm"),
            content=ft.Text("Go home?"),
            actions=[
                ft.TextButton("Yes", on_click=on_dlg_yes),
                ft.TextButton("No", on_click=on_dlg_no),
            ],
            actions_alignment=ft.MainAxisAlignment.END,
        )
        self.page.show_dialog(dlg_modal)
```

Route Templates

```python
import flet as ft

# Using TemplateRoute for parameter matching
troute = ft.TemplateRoute(page.route)

if troute.match("/books/:id"):
    print("Book ID:", troute.id)
elif troute.match("/account/:account_id/orders/:order_id"):
    print("Account:", troute.account_id, "Order:", troute.order_id)
else:
    print("Unknown route")
```

Web URL Strategies

```python
# Path strategy (default): https://myapp.dev/store
# Hash strategy: https://myapp.dev/#/store

ft.run(main, route_url_strategy="hash")
```

---

8. Router System

Flet's Router provides declarative routing inspired by React Router.

Quick Start

```python
import flet as ft

@ft.component
def Home():
    return ft.Text("Welcome home!")

@ft.component
def About():
    return ft.Text("About us")

@ft.component
def App():
    return ft.Router([
        ft.Route(index=True, component=Home),
        ft.Route(path="about", component=About),
    ])

ft.run(lambda page: page.render(App))
```

Route Definitions

Flat Routes

```python
ft.Router([
    ft.Route(index=True, component=Home),
    ft.Route(path="about", component=About),
    ft.Route(path="contact", component=Contact),
])
```

Nested Routes

```python
ft.Route(
    path="products",
    component=ProductsList,
    children=[
        ft.Route(path=":pid", component=ProductDetails),
    ],
)
# /products → ProductsList
# /products/42 → ProductDetails (params: {"pid": "42"})
```

Index Routes

```python
ft.Route(path="settings", children=[
    ft.Route(index=True, component=SettingsHome),   # /settings
    ft.Route(path="profile", component=Profile),     # /settings/profile
])
```

Prefix Routes (Pathless Layout)

```python
ft.Route(component=AdminLayout, children=[
    ft.Route(path="users", component=Users),       # /users
    ft.Route(path="settings", component=Settings), # /settings
])
```

Layout Routes with Outlet

```python
@ft.component
def AppLayout():
    outlet = ft.use_route_outlet()
    return ft.Column([
        ft.AppBar(title=ft.Text("My App")),
        ft.Container(content=outlet, expand=True),
        ft.Text("Footer"),
    ])

ft.Router([
    ft.Route(component=AppLayout, children=[
        ft.Route(index=True, component=Home),
        ft.Route(path="about", component=About),
    ]),
])
```

```

### Dynamic Segments

```python
# Dynamic segment
ft.Route(path="users/:userId", component=UserProfile)
# /users/42 → params: {"userId": "42"}

# Optional segment
ft.Route(path="users/:userId?", component=UserProfile)
# /users → {"userId": None}
# /users/42 → {"userId": "42"}

# Splat (catch-all)
ft.Route(path="files/:path*", component=FileBrowser)
# /files/a/b/c → {"path": "a/b/c"}

# Custom regex
ft.Route(path="item/:id(\\d+)", component=ItemDetails)
# /item/42 → matches
# /item/abc → does NOT match
```

Navigation with Router

```python
# From synchronous callbacks
ft.Button("About", on_click=lambda: ft.context.page.navigate("/about"))

# From async contexts
await page.push_route("/about")
```

Active Links

```python
@ft.component
def NavBar():
    return ft.Row([
        ft.Button(
            "Products",
            style=ft.ButtonStyle(
                bgcolor=ft.Colors.PRIMARY_CONTAINER
                if ft.is_route_active("/products")
                else None,
            ),
            on_click=lambda: ft.context.page.navigate("/products"),
        ),
```
```

```
        ft.Button(
            "About",
            style=ft.ButtonStyle(
                bgcolor=ft.Colors.PRIMARY_CONTAINER
                if ft.is_route_active("/about", exact=True)
                else None,
            ),
            on_click=lambda: ft.context.page.navigate("/about"),
        ),
    ])
```

Data Loading with Loaders

```python
def load_user(params):
    return db.get_user(params["userId"])

ft.Route(path="users/:userId", component=UserProfile, loader=load_user)

@ft.component
def UserProfile():
    user = ft.use_route_loader_data()
    return ft.Text(f"Hello, {user.name}")
```

Authentication Guard

```python
@ft.component
def ProtectedRoute():
    outlet = ft.use_route_outlet()
    if not auth.is_authenticated:
        ft.context.page.navigate("/login")
        return ft.Text("Redirecting...")
    return outlet

ft.Router([
    ft.Route(component=ProtectedRoute, children=[
        ft.Route(index=True, component=Home),
        ft.Route(component=AdminOnly, children=[
            ft.Route(path="admin", component=AdminPanel),
        ]),
    ]),
])
```

404 Not Found

```python
@ft.component
def NotFound():
    location = ft.use_route_location()
    return ft.Text(f"Page not found: {location}")

ft.Router(routes, not_found=NotFound)
```

Multi-View Navigation

```python
# Enable view stack management
ft.Router([
    ft.Route(component=Home, children=[
        ft.Route(path="products", component=ProductsList, children=[
            ft.Route(path=":pid", component=ProductDetails),
        ]),
    ]),
], manage_views=True)

# Component returns View
@ft.component
def ProductDetails():
    params = ft.use_route_params()
    return ft.View(
        route=f"/products/{params['pid']}",
        appbar=ft.AppBar(title=ft.Text(f"Product #{params['pid']}")),
        controls=[
            ft.Text(f"Details for product #{params['pid']}"),
        ],
    )
```

Hooks Reference

Hook	Returns	Description
`use_route_params()`	dict[str, str]	Dynamic segment values from matched route chain
`use_route_location()`	str	Current URL pathname
`use_view_path()`	str	Resolved URL for current view level
`use_route_outlet()`	component	Matched child route component
`use_route_loader_data()`	Any	Return value of current route's loader
`is_route_active(path)`	bool	Whether path matches current location

Recursive Routes

```python
ft.Router([
```

```
    ft.Route(path="folder", component=Folders, children=[
        ft.Route(
            path=":name",
            component=Folder,
            recursive=True,
            children=[
                ft.Route(path="search", component=Search),
            ],
        ),
    ]),
], manage_views=True)
# /folder/a/b/c → stack: [Folders, Folder(a), Folder(a/b), Folder(a/b/c)]
```

---

9. Control Refs

`Ref` utility provides typed references to controls for cleaner code organization.

Without Refs (Direct Variables)

```python
import flet as ft

def main(page):
    first_name = ft.TextField(label="First name", autofocus=True)
    last_name = ft.TextField(label="Last name")
    greetings = ft.Column()

    async def btn_click(e):
        greetings.controls.append(
            ft.Text(f"Hello, {first_name.value} {last_name.value}!")
        )
        first_name.value = ""
        last_name.value = ""
        page.update()
        await first_name.focus()

    page.add(first_name, last_name, ft.Button("Say hello!", on_click=btn_click), greetings)

ft.run(main)
```

With Refs

```python
import flet as ft
```

```
def main(page):
    first_name = ft.Ref[ft.TextField]()
    last_name = ft.Ref[ft.TextField]()
    greetings = ft.Ref[ft.Column]()

    async def btn_click(e):
        greetings.current.controls.append(
            ft.Text(f"Hello, {first_name.current.value} {last_name.current.value}!")
        )
        first_name.current.value = ""
        last_name.current.value = ""
        page.update()
        await first_name.current.focus()

    page.add(
        ft.TextField(ref=first_name, label="First name", autofocus=True),
        ft.TextField(ref=last_name, label="Last name"),
        ft.Button("Say hello!", on_click=btn_click),
        ft.Column(ref=greetings),
    )

ft.run(main)
```

Benefits of Refs

1. **Cleaner Layout**: Controls can be defined inline in `page.add()`
2. **Type Safety**: `ft.Ref[ft.TextField]()` ensures type checking
3. **Better Organization**: Control definitions stay with the layout
4. **IDE Support**: Autocomplete works with `.current`

---

10. Custom Controls

Create reusable UI components by extending Flet controls.

Styled Controls

```python
import flet as ft
from dataclasses import dataclass, field

@ft.control
class MyButton(ft.Button):
    bgcolor: ft.Colors = ft.Colors.ORANGE_300
    color: ft.Colors = ft.Colors.GREEN_800
    style: ft.ButtonStyle = field(
        default_factory=lambda: ft.ButtonStyle(
```

```
            shape=ft.RoundedRectangleBorder(radius=10)
        )
    )
    expand: int = 1

def main(page: ft.Page):
    page.add(
        MyButton(content="OK"),
        MyButton(content="Cancel"),
    )

ft.run(main)
```

Composite Controls

```python
import flet as ft

@ft.control
class Task(ft.Row):
    text: str = ""

    def init(self):
        self.text_view = ft.Text(value=self.text)
        self.text_edit = ft.TextField(value=self.text, visible=False)
        self.edit_button = ft.IconButton(icon=ft.Icons.EDIT, on_click=self.edit)
        self.save_button = ft.IconButton(
            visible=False, icon=ft.Icons.SAVE, on_click=self.save
        )
        self.controls = [
            ft.Checkbox(),
            self.text_view,
            self.text_edit,
            self.edit_button,
            self.save_button,
        ]

    def edit(self, e):
        self.edit_button.visible = False
        self.save_button.visible = True
        self.text_view.visible = False
        self.text_edit.visible = True
        self.update()

    def save(self, e):
        self.edit_button.visible = True
        self.save_button.visible = False
        self.text_view.visible = True
```

```
        self.text_edit.visible = False
        self.text_view.value = self.text_edit.value
        self.update()

def main(page: ft.Page):
    page.add(
        Task(text="Do laundry"),
        Task(text="Cook dinner"),
    )

ft.run(main)
```

Life-Cycle Methods

build()

Called when control is created with `self.page` available.

```python
def build(self):
    # Access self.page here
    self.icon = ft.CupertinoIcons.ADD if self.page.platform == ft.PagePlatform.IOS else ft.Icons.ADD
```

did_mount()

Called after control is added to the page.

```python
def did_mount(self):
    self.running = True
    self.page.run_task(self.update_timer)
```

will_unmount()

Called before control is removed from the page.

```python
def will_unmount(self):
    self.running = False
```

before_update()

Called every time control is updated.

```python
```

```
def before_update(self):
    # Don't call self.update() here
    self.value = f"Count: {self.count}"
```

Isolated Controls

```python
class Task(ft.Row):
    def is_isolated(self):
        return True  # Control updates independently
```

---

11. Declarative vs Imperative

Flet supports both imperative and declarative programming styles.

Imperative Approach

Directly control properties and page tree.

```python
import flet as ft

class Item(ft.Row):
    def __init__(self, first_name, last_name):
        super().__init__()
        self.first_name = first_name
        self.last_name = last_name

        self.first_name_field = ft.TextField(first_name)
        self.last_name_field = ft.TextField(last_name)
        self.text = ft.Text(f"{first_name} {last_name}")
        self.edit_text = ft.Row([self.first_name_field, self.last_name_field], visible=False)
        self.edit_button = ft.Button("Edit", on_click=self.edit_item)
        self.delete_button = ft.Button("Delete", on_click=self.delete_item)
        self.save_button = ft.Button("Save", on_click=self.save_item, visible=False)
        self.cancel_button = ft.Button("Cancel", on_click=self.cancel_item, visible=False)
        self.controls = [
            self.text,
            self.edit_text,
            self.edit_button,
            self.delete_button,
            self.save_button,
            self.cancel_button,
        ]
```

```
def edit_item(self, e):
    self.first_name_field.value = self.first_name
    self.last_name_field.value = self.last_name
    self.text.visible = False
    self.edit_button.visible = False
    self.delete_button.visible = False
    self.save_button.visible = True
    self.cancel_button.visible = True
    self.edit_text.visible = True

def save_item(self, e):
    self.first_name = self.first_name_field.value
    self.last_name = self.last
```